GUC
German University in Cairo

Lecture 4

Square Root Algorithms

CSEN 1038 Advanced Data Structures and Algorithms

Table of contents

Recap

1st Module Summary

- Segment Trees
- Persistent Data Structures
- Treaps
- Square Root Algorithms ←

Subarray Sum Problem Revisited

Subarray sum: Problem Framework

Applying the standard query-based structure to this problem looks like this:

1. Initialization: You are given an initial array of n numerical elements, often represented as $A = [a_1, a_2, \ldots, a_n]$.
2. Support Updates: The system must handle changes to the data, such as updating the value of an element at a specific index i (Point Updates) or modifying a range of values (Range Updates).
3. Support Queries: The core task is to quickly calculate the sum of elements within a given range $[L, R]$, defined as:

$$\sum_{i=L}^{R} a_i$$

Batch Processing

1st Algorithm: Batch Processing

This technique addresses the point of avoiding performing the updates at the moment we receive them!

It balances the complexity between updates and queries.

Block Decomposition

2nd Algorithm: Block Decomposition

Block Decomposition answers the following question:

Can I calculate the answer for some subarrays and use them to answer the queries?

This technique is so powerful as it allows answering complicated queries and supports updates as well.

(Bonus) K-Query Revisited

K-Query: Problem Framework

1. **Initialization**
   The system is provided with an initial, fixed array A of n integers:

$$A = [a_1, a_2, \ldots, a_n]$$

2. **Update Operations**
   None: The problem is defined on a static dataset.
3. **Query Execution**
   The core task is to process multiple queries of the form $[l, r, k]$. For each query, the system must calculate the count of elements within the specified subarray that are strictly greater than a threshold k.

Formally, given the parameters $1 \leq l \leq r \leq n$ and an integer k, the result is the cardinality of the set:

$$\text{Result} = |\{i \mid l \leq i \leq r \text{ and } a_i > k\}|$$

Case Processing

3rd Algorithm: Case Processing

This technique attempts to combine two algorithms that complete one another into one **piecewise** algorithm!

Depending on the case of the query we run the appropriate algorithm.

Subarray Step Sum Problem

Subarray Step Sum: Problem Framework

1. **Initialization**
   The system is provided with an initial array A of n numerical elements:
   $$A = [a_1, a_2, \ldots, a_n]$$
2. **Update Operations**
   None: This problem is defined on a static dataset.
3. **Query Execution**
   The core task is to process queries of the form $\{l, r, s\}$, where $1 \leq l \leq r \leq n$ and $s \geq 1$. For each query, the system calculates a sum by traversing the array backwards from r to l in steps of size s.

Questions?

References i

- ultimate topic list:
  https://youkn0wwho.academy/topic-list/?category=data_structures&subCategory=sqrt_decomposition
- guc community session:
  https://youtu.be/D0Yr9xticx8?si=OJCLEc--uIDVC__b